Deterministic, Instruction-Free Multi-Model Inference on a Configurable Spatial Dataflow Fabric for Real-Time Edge Systems

Kulamani Rout · B.Tech EEE, IIIT Bhubaneswar · routsarojini901@gmail.com · github.com/kulamani007

Research proposal for PhD admission

1. Overview

Edge systems such as nano-UAVs, AR devices, and smart sensors must run several neural networks at once (for example, obstacle detection, visual odometry, and audio-event detection) under hard deadlines and tight power budgets. Today's edge NPUs are instruction-driven: a processor or microcontroller fetches, decodes, and dispatches every layer and tile. This keeps them flexible, but control overhead, dynamic arbitration, and shared-memory contention make latency hard to predict. The alternative, fixed-function model-specific ASICs, is efficient and predictable but cannot be reprogrammed.

I propose to study a middle point: **a fixed silicon fabric of configurable dataflow stages in which no general-purpose instruction processor participates in the inference loop.** A compiler transforms each model graph into hardware configuration, routing, memory allocation, and a static schedule. Dedicated dataflow stages then execute inference directly on streaming inputs. Because the complete execution schedule is known before inference begins, the compiler can **compute a worst-case latency bound for every model, including when several models share the fabric concurrently**, while the fabric stays reprogrammable for new models at load time.

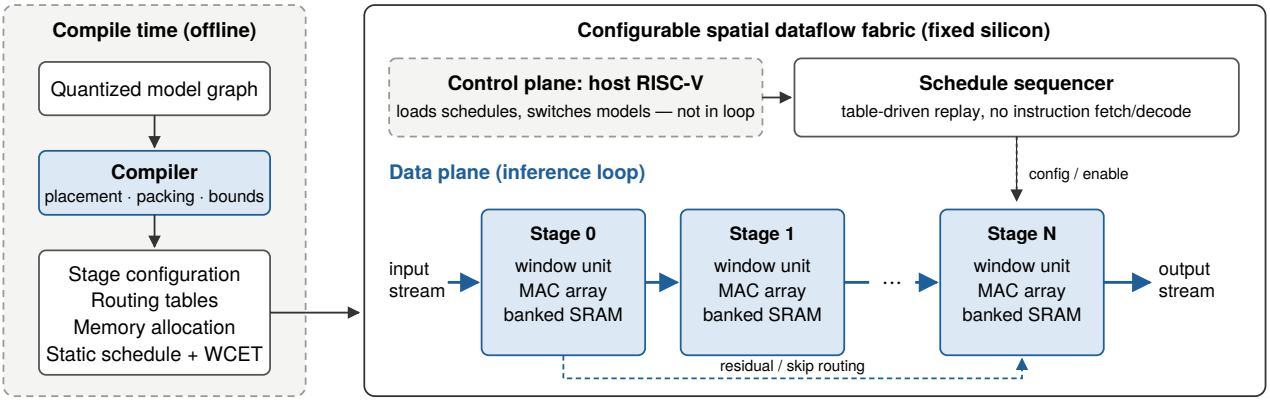


Figure 1. Proposed execution model. The compiler produces all configuration and a static schedule offline; at run time, streaming data flows through configurable stages with no instruction processor in the inference loop.

The central question is whether determinism at the hardware level can be obtained without giving up the flexibility and efficiency of a programmable NPU, and where the boundary between the two lies.

2. Research Questions

RQ1 (Determinism). Can a configuration-driven, instruction-free fabric give compile-time worst-case latency bounds for concurrently executing models that are tight (within a small margin of measured latency) and hold under memory contention?

RQ2 (Efficiency). Compared with an equal-resource, instruction-driven NPU (same MAC count, on-chip SRAM, and DRAM bandwidth), what does removing per-layer dispatch save in control energy and latency, and what does fixing the stage structure cost in utilization?

RQ3 (Mapping). How should a compiler jointly choose, per layer, spatial placement, temporal reuse of a stage, bottleneck-layer replication, on-chip memory partitioning, and multi-model packing so that deadlines are met with minimum energy?

3. Architecture

3.1 Control plane vs. data plane

- **Data plane (the inference loop):** configurable dataflow stages connected by a routing network. Each stage has a parameterized INT8/INT4 MAC array, banked local SRAM, a window-extraction unit, vector support for activation, normalization, and residual add, and double-buffered configuration registers. Stages communicate through ready/valid streams. There is no instruction fetch or decode.
- **Schedule sequencer:** a small, non-programmable, table-driven unit that replays the compiler's static schedule (stage activation windows, weight-load points, reconfiguration points).
- **Control plane (outside the loop):** a host RISC-V core that loads compiled schedules, switches model contexts between inference windows, and handles exceptions. It is never invoked per layer or per tile.
- Operators outside the supported set (convolution, depthwise convolution, matrix multiply, activation, normalization, residual add, pooling) are rejected at compile time or confined to graph boundaries such as pre- and post-processing. They are never interleaved mid-pipeline.

3.2 Window-extraction unit (preliminary work)

Each stage must deliver a full M×N input window per cycle for any stride. Following established memory-partitioning theory [11], I store feature maps in a two-dimensional cyclic partition of R×C single-port SRAM banks: pixel (r, c) is placed in bank (r mod R, c mod C). Any M×N window with M ≤ R and N ≤ C then touches each bank at most once, so a full window is read in one cycle. Each bank's address is a base address plus a one-bit carry determined by the window's start offset, and a small rotation network restores window order. The unit is parameterized by R, C, M, N, and stride, and a **multi-hot bank-enable mask** is programmed by the compiler.

A Python functional model has verified conflict-free, bit-exact window extraction for 3×3, 5×5, 3×5, and 1×1 windows at stride 1 and stride 2, on feature maps up to 58×58. It also shows why the enable mask matters: on a 5×5 bank grid provisioned for the largest kernel, a 3×3 layer activates only 36% of the banks and a 1×1 layer only 4%. The compiler-programmed multi-hot mask lets the unused banks be clock-gated. This builds on my earlier work on clock-gated MAC arrays and ICG-based ALU clock gating.

Address generation. A window-address generator replaces instruction-driven address computation. It holds the window's top-left base address (r0, c0), advances it by the configured stride, and wraps to the next window row at the end of each row. All M×N pixel addresses are derived from this base, and the bank mapping above turns them into one parallel read per cycle. Stride, kernel size, and feature-map dimensions are configuration values written by the compiler, so no processor is involved while the window slides.

Fetch modes. Consecutive windows overlap, so re-reading every window in full wastes SRAM energy. The unit supports two modes:

- **FULL:** fetch all M×N pixels per window. This allows any stride and any traversal order.
- **SHIFT:** keep the current window in a register and fetch only the min(s, N) new columns on each step.

Both modes are bit-exact in the functional model. On the 3×3 layers of MobileNetV1, SHIFT reduces window-fetch SRAM reads by 57% overall:

MobileNetV1 3×3 layers (all channels)	FULL reads	SHIFT reads	Reduction
Stride 1 (9 layers)	14.0 M	5.0 M	64%
Stride 2 (5 layers)	3.7 M	2.5 M	32%
All 3×3 layers	**17.7 M**	**7.6 M**	**57%**

The saving drops to zero whenever the stride is at least the kernel width (2×2 pooling at stride 2, strided 1×1 convolution), because consecutive windows no longer overlap. For those layers the compiler bypasses and clock-gates the shift register, and FULL mode stays available wherever a layer's traversal order is not row-major (for example, tiled or interleaved multi-model schedules). Extending reuse vertically with line buffers, and measuring the area cost of the variable-shift register, are next steps.

Cyclic memory partitioning, strided address generation, and sliding-window registers are each established techniques. My contribution is not these primitives. It is a **fixed-silicon, compiler-configurable** combination of them, in which the compiler selects fetch mode, bank gating, and address parameters per layer, integrated into a processor-free stage.

4. Methodology

1. **Workload characterization.** Select heterogeneous edge workloads: a detection CNN, a keyword-spotting model (DS-CNN from MLPerf Tiny), and a compact transformer (KWT or MobileViT). Measure operator mix, tensor shapes, memory traffic, and per-layer latency.
2. **Cycle-level simulator and latency analyzer.** Build a parameterized model of stage count, MAC lanes, bank grid, routing, SRAM, and DRAM bandwidth, together with a static analyzer that computes per-model worst-case latency from a compiled schedule.
3. **Compiler.** Import quantized ONNX or TFLite graphs, apply fusion and tiling, perform placement, routing, memory partitioning, replication, and multi-model packing, and emit configuration and schedule tables. Placement is formulated as a constrained optimization problem (deadlines plus resources), with a heuristic for larger models.
4. **FPGA prototype.** Implement a 4–8 stage fabric with the window-extraction unit, residual routing, and the schedule sequencer. Validate functional correctness, cycle counts against the simulator, frequency, and resource cost.
5. **Evaluation.**
   - *Baselines:* an equal-resource, instruction-driven accelerator (Gemmini [12] with a RISC-V host), a layer-sequential monolithic array, and static 50/50 multi-model partitioning.
   - *Metrics:* throughput; mean and tail latency; bound-vs-measured latency gap; energy per inference and energy-delay product; utilization; DRAM traffic; reconfiguration overhead; accuracy after quantization.
   - *Energy method:* energy and area are estimated through an ASIC flow (ASAP7 or SKY130 with open-source tools) together with an architectural energy model, because FPGA power does not represent fixed-silicon costs.

5. Expected Contributions

1. A processor-free, configurable spatial dataflow fabric whose inference latency is statically analyzable.
2. A compiler that jointly performs placement, temporal reuse, replication, memory partitioning, and multi-model packing under deadline constraints, together with a worst-case latency analyzer.
3. A compiler-configured window-extraction unit (stride-driven address generation, bank-partitioned single-cycle fetch, and per-layer selection between full-fetch and shift-reuse modes with clock gating of unused resources) for variable kernel size and stride.

4. A quantitative characterization of when instruction-free execution pays off against instruction-driven NPUs at edge scale, and when it does not.

6. Positioning Against Prior Work

- **Layer pipelining and inter-layer dataflow** (fused-layer CNN accelerators [1], Tangram [2], SET [3]) reduce memory traffic but target throughput and energy, not guaranteed latency.
- **Multi-tenant accelerators** (Planaria [4], Herald [5], Zeng et al. [6]) share hardware across models through dynamic arbitration, which improves utilization but weakens predictability.
- **Streaming FPGA generators** (FINN [7], hls4ml [8], fpgaConvNet [9]) avoid instruction processors but produce a new bitstream per model, so they cannot be reprogrammed in fixed silicon.
- **Real-time DNN scheduling** on CPU/GPU platforms [10] and deadline-aware FPGA preemption provide guarantees at the software or OS level on top of non-deterministic hardware.

This work targets the gap between these lines: **determinism provided by the hardware and the compiler themselves, on reprogrammable fixed silicon, for concurrent models.**

7. Plan (3–4 years)

Year	Focus
1	Workload study, simulator, latency analyzer, window-extraction RTL; first workshop paper
2	Compiler (placement, packing, bounds); FPGA prototype with 4–8 stages; conference paper
3	Multi-model deadline scheduling, ASIC-flow energy/area evaluation, real edge case study (e.g., nano-UAV perception)
4	Generalization to transformer blocks, thesis

8. Relevant Background

- Collaborative research at the University of Tsukuba on FPGA-based processing for nano-UAVs.
- Deployment of MobileNet on the Coral NPU (a RISC-V-based, instruction-driven edge NPU), which provides direct experience with the class of baseline this work compares against.
- RTL design of clock-gated MAC arrays and ICG-based ALU clock gating.

Selected References

1. M. Alwani, H. Chen, M. Ferdman, P. Milder, “Fused-Layer CNN Accelerators,” MICRO 2016.
2. M. Gao et al., “Tangram: Optimized Coarse-Grained Dataflow for Scalable NN Accelerators,” ASPLOS 2019.
3. J. Cai et al., “Inter-layer Scheduling Space Definition and Exploration for Tiled Accelerators” (SET), ISCA 2023.
4. S. Ghodrati et al., “Planaria: Dynamic Architecture Fission for Spatial Multi-Tenant Acceleration of Deep Neural Networks,” MICRO 2020.
5. H. Kwon et al., “Heterogeneous Dataflow Accelerators for Multi-DNN Workloads” (Herald), HPCA 2021.
6. S. Zeng et al., “Serving Multi-DNN Workloads on FPGAs: A Coordinated Architecture, Scheduling, and Mapping Perspective,” IEEE Trans. Computers, 2023.
7. Y. Umuroglu et al., “FINN: A Framework for Fast, Scalable Binarized Neural Network Inference,” FPGA 2017.
8. J. Duarte et al., “Fast Inference of Deep Neural Networks in FPGAs for Particle Physics” (hls4ml), JINST, 2018.
9. S. I. Venieris, C.-S. Bouganis, “fpgaConvNet: A Framework for Mapping CNNs on FPGAs,” FCCM 2016.
10. Y. Xiang, H. Kim, “Pipelined Data-Parallel CPU/GPU Scheduling for Multi-DNN Real-Time Inference” (DART), RTSS 2019.
11. Y. Wang, P. Li, J. Cong, “Theory and Algorithm for Generalized Memory Partitioning in High-Level Synthesis,” FPGA 2014.
12. H. Genc et al., “Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration,” DAC 2021.
13. M. Abdelfattah et al., “DLA: Compiler and FPGA Overlay for Neural Network Inference Acceleration,” FPL 2018.